

STUDENT WORKBOOK



Python for Young Innovators

From First Program to Your First AI Project

A complete 11-week Python and Machine Learning course for secondary school students

Mr. Muwatta

Founder, Algorise Tech Explorers | www.muwatta.com.ng

Now live online at muwatta.com.ng/academy

Contents

PART 1, COURSE OVERVIEW

- Setup Guide (Do this once, before Week 1)

PART 2, 11-WEEK CURRICULUM MAP

PART 3, COMPLETE STUDENT COURSE MATERIAL

- 1 Starting to Code
- 2 Making Decisions
- WEEKS 3–11, Continuing the Course
- PHASE 3, PYTHON FOR DATA
- PHASE 4, INTRODUCTION TO AI AND MACHINE LEARNING
- PHASE 5, BUILDING AI/ML PROJECTS
- PHASE 6, INDEPENDENT FINAL PROJECT

PART 4, EXERCISES AND SOLUTIONS

PART 5, PROJECTS

- Weekly Mini-Projects
- Final Project Framework (for all Final Project options)
- Final Project Options (choose one, or propose your own using the same framework)

PART 6, ASSESSMENTS

- Weekly Exercises & Quizzes
- Sample Mini-Quiz Format (used at the start of Sessions, 5 minutes)
- Coding Challenges
- Mid-Course Practical Assessment (End of Week 6)
- Final Project Assessment
- Final-Project Rubric (100 points total)

PART 7, FINAL PROJECT READINESS MATRIX

A Note From Mr. Muwatta

Welcome to this course. My name is Mr. Muwatta, and I am the founder of Algorise Tech Explorers. I built this course because I believe that every student, no matter their background or their current comfort with computers, can learn to code and can go on to build real Artificial Intelligence and Machine Learning projects, as long as the material is explained clearly, in order, with enough practice at every step.

You do not need to be a “maths genius” or already know what programming looks like to succeed here. You only need to be willing to type code, make mistakes, read what the computer tells you, and try again. That is exactly how I expect you to learn, and it is exactly how real programmers and AI engineers work every single day.

By the end of Week 11, you will have built your own small Machine Learning project, from choosing a problem all the way to making a real prediction and explaining it to someone else. I am proud to guide you there.

Visit Algorise Academy online. This course now has an online home at muwatta.com.ng/academy, where you can follow these same lessons, track your progress, and try out your code interactively. Use this workbook alongside the Academy, or on its own, whichever works best for you.

Mr. Muwatta

Founder, Algorise Tech Explorers
www.muwatta.com.ng

PART 1, COURSE OVERVIEW

Course Title: Python for Young Innovators, From First Program to Your First AI Project

Target Students: Basic 4, Basic 5, JSS1, JSS2, JSS3 (ages approx. 9–15), ranging from complete beginners to students with light prior exposure to programming.

Duration: 11 weeks

Total Hours: 44 hours (2 sessions/week × 2 hours × 11 weeks)

Prerequisites: None. No prior programming, no advanced mathematics, no command-line experience required. Students need a laptop or desktop computer (Windows, Mac, or Linux) able to run Python.

Learning Philosophy: This course is practical-first, incremental, and project-driven. Students write code in nearly every session from Week 1 onward. Every concept is explained from zero, nothing is assumed. The course is designed backward from one final test: could you keep going and finish your project using only this material, even on a week you worked entirely on your own? Every skill needed for the final AI/ML project is taught and practiced well before you need it. Scaffolding, the amount of help given, is heavy at the start and is deliberately removed week by week, so that by Weeks 10–11 students are designing and defending their own project decisions.

Final Learning Outcomes (by the end of Week 11, every student should be able to):

1. Write, run, and debug Python programs confidently.
2. Explain what a variable, loop, function, and error message are, in their own words.
3. Work with lists, dictionaries, and simple text files of data.
4. Load a small dataset with Pandas and describe what its rows and columns represent.
5. Explain the difference between a feature and a target/label.
6. Explain, in plain language, what “training a model” means.
7. Use scikit-learn to train a simple classification or regression model.
8. Test a model and describe how good or bad its predictions are, in simple terms.
9. Use a trained model to make a new prediction and interpret the result.
10. Modify an existing project (change a feature, change a dataset, change a value) and explain what changed and why.
11. Explain their finished project to another person without reading from a script.

Your Path to Becoming an AI/ML Engineer:

Everything in this workbook is aimed at one destination: helping you take real, honest first steps toward becoming an Artificial Intelligence and Machine Learning engineer. That may sound like a huge, distant title right now, and that is completely normal. But an AI/ML engineer is simply a person who has learned to do a small number of things well, and this course teaches you every one of them, at a beginner’s scale, in order.

In simple words, an AI/ML engineer’s daily work usually involves five things:

- **Understanding a real problem.** Someone needs a question answered, will this student pass, is this photo a cat or a dog, will this machine break down soon, and the engineer has to understand that problem clearly before touching any code.
- **Collecting and cleaning data.** Real data is messy. It has missing pieces, mistakes, and strange values. Engineers spend enormous amounts of time simply making data usable.
- **Choosing and training a model.** Once the data is ready, the engineer chooses a method that can learn patterns from it, and trains that method on real examples.
- **Testing whether it actually works.** A model that looks good on paper can still be wrong. Engineers test carefully, using data the model has never seen, before trusting it.
- **Explaining the result to other people.** A brilliant model that nobody understands is not very useful. Engineers must explain, in plain language, what their system does and does not do well.

By the end of Week 11, you will have personally done all five of these things yourself, on your own small project. That is not a simplified imitation of the job, it is genuinely the same loop, at a smaller and gentler scale. Every week of this course, including the “Career Corner” boxes you will see throughout, is designed to connect what you are practicing directly back to this real profession, so that by the time you reach harder, more advanced material later in life, the foundation will already feel familiar rather than frightening.

Tools / Software Required:

- A laptop or desktop computer (no special hardware; an ordinary machine is enough).
- Python 3 (installation covered in the Setup Guide below).
- A simple code editor, we recommend **Thonny** for beginners (built-in, beginner-friendly, no configuration) or **VS Code** for students who want something closer to professional tools.
- Internet access is helpful but **not required for every lesson**, installation happens once, and after that almost everything runs offline. Datasets used in this course are small, and offline copies (as `.csv` files) are provided or can be typed/created by hand.
- Libraries used, introduced gradually: `pandas` , `numpy` , `matplotlib` , `scikit-learn` (all free and open-source).

Setup Guide (Do this once, before Week 1)

1. What is Python? Python is a programming language, a way of writing instructions that a computer can follow exactly, one step at a time. We use Python because it reads almost like plain English and is one of the most widely used languages for AI and Machine Learning in the real world.

2. How to install Python

- Go to python.org/downloads (do this with an adult, or with me, if you're not sure).
- Download the version for your operating system (Windows or Mac).
- Run the installer. **Important (Windows only):** on the first install screen, tick the box that says “Add Python to PATH” before clicking Install.
- Finish the installation.

3. How to check whether Python is installed Open a terminal (Windows: search for “Command Prompt”; Mac: search for “Terminal”) and type:

```
python --version
```

If that doesn't work, try:

```
python3 --version
```

You should see something like `Python 3.12.1` . If you see an error, ask me, Mr. Muwatta, or another adult helping you, usually it means Python wasn't added to PATH during install.

4. Install Thonny (recommended editor for this course)

- Go to thonny.org, download the version for your system, and install it like any other program.
- Open Thonny. You'll see a white script area at the top and a “Shell” area at the bottom, that's where output and errors appear.

5. How to create a project folder

- On your computer, create a folder called `python-course` somewhere easy to find, such as your Desktop.
- Inside it, you will create a new folder for each week, e.g. `week1` , `week2` .

6. How to create a Python file

- In Thonny: File → New. Save it (File → Save) inside your week's folder with a name ending in `.py` , for example `lesson1.py` .

7. How to run Python code

- In Thonny, click the green **Run** button (or press F5). The output appears in the Shell area below.

8. How to install libraries When we reach Week 6 you will need extra libraries. In Thonny: **Tools** → **Manage Packages**, then search for the library name (e.g. `pandas`) and click **Install**. If using a terminal instead:

```
pip install pandas numpy matplotlib scikit-learn
```

If `pip` is not recognized, try `pip3` instead of `pip` .

9. What to do if installation fails

- Restart your computer and try again.
 - Make sure you have internet access for this one-time step.
 - Ask me to install it together during class, this is normal and not a sign you did anything wrong.
-

PART 2, 11-WEEK CURRICULUM MAP

Week	Main Topic	Python Skills	AI/ML Connection	Practical Project	Expected Outcome
1	Starting to Code	print, comments, variables, data types, input	Data must be <i>stored</i> before it can be used by anything, including AI	“About Me” and “Mini Calculator” programs	Student can write, save, and run a working Python program
2	Making Decisions	operators, comparisons, if/elif/else, logical operators	AI systems constantly make decisions based on conditions in data	“Pass or Fail” and “Weather Advisor” programs	Student can write branching programs
3	Repeating & Organizing Data	for loops, while loops, lists, indexing, basic debugging	Real data comes in large collections, loops let us process every item, just like a model processes every row	“Class Score Tracker”	Student can loop through and summarize a list of numbers
4	Reusable Code & Dictionaries	functions, parameters, return values, dictionaries	Models are reusable “functions” that take inputs and give outputs; dictionaries store labeled data like rows in a dataset	“Student Record” program using functions + dictionaries	Student can write and call their own functions
5	Files & Errors	reading/writing text files, basic exception handling, review	Datasets live in files; real programs must handle bad data without crashing	“Save and Load Scores” program	Student can read/write a file and handle a simple error
6	Meet Your Data	what is a dataset, rows/columns, features/labels, reading CSV with Pandas	This is the literal shape data must be in for ML	Load and explore a small CSV (e.g. class attendance or exam scores)	Student can open a CSV and describe its rows/columns
7	Exploring Data	Pandas basics, NumPy basics, simple Matplotlib charts	“Exploring data” is the first real step of every ML project	Build a simple chart from real data (e.g. average scores per subject)	Student can compute averages and plot a simple chart
8	What Is AI & ML, Really?	no new syntax, concept week, light guided code	Core mental model: Traditional programming vs ML; features, labels, training, prediction	“Human as a Model” unplugged activity + guided demo	Student can explain in their own words what a model is
9	Your First Model	scikit-learn basics: train/test split, <code>.fit()</code> , <code>.predict()</code> , accuracy	This is the actual mechanics of building an ML model	Guided classification project (e.g. Iris flowers, or pass/fail prediction)	Student can train and test a real model with guidance
10	Build With Less Help	choosing features/target, evaluating a model, experimenting	Bridge from guided to independent build	Guided-but-lighter regression or classification project of student’s choice from a list	Student can adapt the Week 9 workflow to a new dataset
11	Your Own Project	independent project build, debugging, presenting	Full pipeline, done independently	Final Project (from Part 5)	Student can build, explain, and present a small ML project alone

PART 3, COMPLETE STUDENT COURSE MATERIAL

How to use this material: Each week has two sessions. Every session follows the same shape: warm-up → explanation → guided coding → practice → challenge → wrap-up. If you missed a class, start from “What You Will Learn” and work through in order, everything you need is explained here, not just referenced.

1 Starting to Code

GOAL Weekly Goal

By the end of this week, you will be able to write a Python program that stores information in variables, prints messages, and asks the user questions, completely from scratch.

CAREER Career Corner: On the Job as an AI/ML Engineer

Every AI and Machine Learning engineer working today began exactly where you are right now: writing tiny, simple lines of code and learning to think in exact, patient steps. Before anyone builds a self-driving car’s vision system or a chatbot that talks to millions of people, they spend months becoming comfortable with the basics you are learning this week, how to store information, give clear instructions, and read error messages without panicking. There is no shortcut past this stage, and every professional you admire once sat exactly where you are sitting now.

SESSION 1, What Is Programming? Your First Lines of Python

GOAL 1. What You Will Learn

By the end of this lesson, you should be able to:

- explain what a computer program is
- explain what Python is and why we’re using it
- write and run a program using `print()`
- add comments to explain your code

2. Why Does This Matter?

A computer cannot guess what you want, it only does exactly what you tell it, in an order it understands. Programming is the skill of giving a computer clear, exact instructions. Every AI system, no matter how advanced, is built out of very ordinary instructions like the ones you’ll write today. If you can’t yet imagine how “AI” connects to typing text into a black window, that’s completely normal. By Week 8 it will make sense. For now, focus on becoming comfortable giving the computer instructions.

FACT Did You Know?

Python is the single most widely used programming language among professional AI and Machine Learning engineers worldwide. It was not originally designed for AI at all, it was created in 1991 to be easy to read and easy to teach. That very quality, being easy for humans to read, is exactly why it later became the favorite language for building intelligent systems.

WORDS 3. Key Words

- **Program:** A set of instructions that tells a computer what to do.
- **Python:** The programming language we are using in this course.
- **Code:** The instructions you write, in a language the computer understands.
- **Run:** To make the computer actually follow your instructions.
- **Comment:** A note in your code that Python ignores, it’s there for humans to read.

LEARN 4. Learn the Concept

A computer only does what you tell it, step by step, from top to bottom. Python is one way of writing these instructions.

One of the simplest instructions in Python is `print()`. It tells the computer: “Show this text to the person using the program.”

```
print("Hello, World!")
```

Here, `print` is a **function**, a built-in command that does something (we’ll learn to build our own functions in Week 4). The round brackets `()` hold what we want it to act on. The text inside quotation marks `"Hello, World!"` is called a **string**, Python’s word for a piece of text.

You can also leave notes for yourself (or anyone else reading your code) using a `#`. Python ignores anything after a `#` on that line:

```
# This program greets the world
print("Hello, World!") # this line displays a message
```

EX 5. First Example

```
# My first Python program
print("Hello, World!")
print("I am learning Python.")
```

Line-by-line:

- Line 1 is a comment, Python skips it completely.
- Line 2 tells the computer to display the text `Hello, World!`.
- Line 3 tells the computer to display a second message, on a new line.

Run it. You should see:

```
Hello, World!
I am learning Python.
```

TRY 6. Try It Yourself

Write a program that prints your name, your school, and one thing you’re hoping to learn in this course, three separate `print()` lines.

HINT 7. Hint

You need three separate `print()` lines, one under the other. Each piece of text goes inside quotation marks.

CHECK 8. Check Yourself

1. What symbol does Python use to start a comment?
2. What will this print?

```
print("2 + 2")
```

(Careful, read it exactly as written.)

3. Why do you think the text in `print()` needs quotation marks?

DEBUG 9. Debugging Challenge

```
print("Hello, World!"
```

What is wrong with this program? *Hint: count the brackets.* (Don’t check the fix yet, try running it yourself first and read what Python tells you.)

CHALLENGE 10. Mini Challenge

Problem: Write a short “About Me” program.

Requirements: At least 4 `print()` lines, including your name, age, favorite subject, and one comment explaining what the program does.

HINT Hint

Start with a comment line, then your four `print()` statements.

Optional harder challenge: Add a blank print line (`print()` with nothing inside) between two of your lines to create spacing.

AI 11. AI/ML Connection

Every AI system starts the same way this program did: with instructions a computer can follow exactly. Later, we'll write programs that don't just print fixed text, they will look at data and decide what to print based on patterns. But the starting skill is the same one you just used: telling the computer, precisely, what to do.

PRACTICE 12. Practice Tasks

- **Easy:** Print your favorite food and favorite color, one per line.
- **Easy:** Print a 3-line "poem" (it doesn't need to rhyme).
- **Medium:** Print a small "menu" for a fictional shop: 3 items with prices, each on its own line.
- **Medium:** Add comments above each `print()` line in your menu program explaining what it does.
- **Challenge:** Use `print()` to draw a simple shape using text characters, e.g.:

```
*  
**  
***
```

EXIT 13. Exit Ticket

1. What does `print()` do?
2. What is a comment used for?
3. What would happen if you forgot a closing quotation mark?
4. Name one thing you found confusing today.

14. What You Should Be Able to Do Now

- ☐ I can open Thonny and create a new Python file.
- ☐ I can use `print()` to display text.
- ☐ I can write comments in my code.
- ☐ I can run my program and see the output.

SESSION 2, Variables and Data Types**GOAL** 1. What You Will Learn

By the end of this lesson, you should be able to:

- create a variable and store a value in it
- change a variable's value
- use a variable inside `print()`
- identify text (string), whole number (integer), decimal number (float), and True/False (boolean) values
- use `input()` to get information from the user

2. Why Does This Matter?

So far we only printed fixed text. But real programs, and every AI system, need to **store** information so they can use it later, combine it, or change it. A variable is how we store a single piece of information with a name we choose. Machine learning is, at its core, a way of finding patterns in huge collections of stored values. Today you learn the smallest unit of that: one variable holding one value.

WORDS 3. Key Words

- **Variable:** A name we use to store a value.
- **Value:** The actual information stored.
- **String:** Text data, written in quotes, e.g. `"Ada"`.
- **Integer (int):** A whole number, e.g. `12`.
- **Float:** A decimal number, e.g. `12.5`.
- **Boolean (bool):** Either `True` or `False`.
- **Input:** Information typed in by the person using the program.

LEARN 4. Learn the Concept

A variable is created like this:

```
name = "Abdullahi"
```

`name` is the variable, a label we chose. `"Abdullahi"` is the value stored inside it. The `=` sign doesn't mean "equals" the way it does in maths, it means "store this value in this name."

Variables can hold different **types** of data:

```
name = "Abdullahi"    # string (text)
age = 12               # integer (whole number)
height = 1.45          # float (decimal number)
is_student = True      # boolean (True or False)
```

You can print a variable directly:

```
print(name)
```

Or combine text and variables, separated by commas:

```
print("My name is", name, "and I am", age, "years old.")
```

You can also ask the user to type something in, using `input()`. Whatever the user types is always stored as a **string**, even if it looks like a number:

```
name = input("What is your name? ")
print("Hello,", name)
```

EX 5. First Example

```
name = "Abdullahi"
age = 12
print("My name is", name)
print("I am", age, "years old")
age = 13
print("Next year I will be", age)
```

Line-by-line:

- Lines 1–2 create two variables and store values in them.
- Line 3 prints text combined with the `name` variable.
- Line 4 prints text combined with the `age` variable.
- Line 5 **changes** the value stored in `age`, the old value (12) is replaced.
- Line 6 shows the new value.

Output:

```
My name is Abdullahi
I am 12 years old
Next year I will be 13
```

TRY 6. Try It Yourself

Write a program that:

1. Creates variables for your name, age, and favorite subject.
2. Prints a sentence using all three variables in one `print()` line.
3. Uses `input()` to ask the user for their own name, and greets them using it.

HINT 7. Hint

You'll need one variable per piece of information, and you can list several things inside one `print()` separated by commas.

CHECK 8. Check Yourself

1. What will this print?

```
score = 70
print(score)
score = 85
print(score)
```

2. What data type is `"12"`, is it the same as `12`? Why might this matter?
3. If you use `input()` to ask someone's age, what type will Python store it as?

DEBUG 9. Debugging Challenge

```
Name = "Zainab"
print(name)
```

What is wrong with this program? *Hint: Python is case-sensitive, look very closely at both lines.*

CHALLENGE 10. Mini Challenge

Problem: Build a small "Student Profile" program.

Requirements: Ask the user for their name, age, and favorite subject using `input()`, store each answer in a variable, then print a full sentence introducing them using all three variables.

HINT Hint

You'll need three `input()` lines and one combined `print()` line at the end.

Optional harder challenge: Also ask for their score in a subject (as a number using `input()`), and print it back to them, adding the word "points" after it.

AI 11. AI/ML Connection

Think of a spreadsheet of exam scores: each student's name, age, and score are stored somewhere. A variable is the smallest possible version of that idea, one labeled box holding one piece of information. Later, we'll use **lists** and **dictionaries** to store many of these at once (Weeks 3–4), and eventually whole **datasets** with thousands of rows (Week 6), but it all starts with a single variable.

PRACTICE 12. Practice Tasks

- **Easy:** Create three variables (name, favorite animal, favorite number) and print them in one sentence.
- **Easy:** Ask the user for their age using `input()` and print "You are ____ years old."
- **Medium:** Create variables for a book's title, author, and number of pages, and print a short "book card."
- **Medium:** Ask the user for two numbers using `input()`, and print both of them back labeled clearly (don't add them yet, that's Week 2).

- **Challenge:** Create a “Fact File” about an animal of your choice using at least one string, one integer, one float, and one boolean variable (e.g. `is_endangered = True`), and print all of them in a readable way.

EXIT 13. Exit Ticket

1. What is the difference between a variable’s name and its value?
2. Name the four data types you learned today.
3. What type does `input()` always give you?
4. What would happen if you tried to use a variable before creating it?

14. What You Should Be Able to Do Now

- ☐ I can create a variable.
- ☐ I can store text, whole numbers, decimals, and True/False in variables.
- ☐ I can print a variable, and combine variables with text in one `print()`.
- ☐ I can change a variable’s value.
- ☐ I can use `input()` to collect information from the user.

2 Making Decisions

GOAL Weekly Goal

By the end of this week, you will be able to write programs that compare values and make different decisions depending on the situation.

CAREER Career Corner: On the Job as an AI/ML Engineer

AI and Machine Learning engineers spend a large part of their working day writing conditions: deciding what a program should do when the data looks one way versus another. Later in your journey, when you build a rule such as “if the model’s confidence is above 80 percent, trust its prediction, otherwise ask a human to check it,” you will be using exactly the `if`, `elif`, and `else` skills you are practicing this week. Decision-making code like this sits underneath almost every real AI system, quietly making choices thousands of times per second.

SESSION 1, Operators and Comparisons

GOAL 1. What You Will Learn

- use arithmetic operators (`+` `-` `*` `/` `//` `%`) to calculate values
- convert between data types using `int()`, `float()`, `str()`
- compare two values using `==` `!=` `>` `<` `>=` `<=`
- understand that comparisons produce a boolean (`True` / `False`)

2. Why Does This Matter?

Decisions in programming, and in AI, are built entirely from comparisons. “Is this score higher than 50?” “Is this temperature above freezing?” Every one of those questions becomes a boolean. Before we can make our programs decide anything (next session), we need to be fluent in asking exact, comparable questions about our data.

WORDS 3. Key Words

- **Operator:** A symbol that performs an action on values, e.g. `+`.
- **Type conversion:** Changing a value from one data type to another.
- **Comparison operator:** A symbol that compares two values and returns `True` or `False`.

LEARN 4. Learn the Concept

Arithmetic operators:

```
print(5 + 3)    # addition -> 8
print(5 - 3)    # subtraction -> 2
print(5 * 3)    # multiplication -> 15
print(5 / 3)    # division -> 1.666...
print(5 // 3)   # whole-number division -> 1
print(5 % 3)    # remainder ("modulo") -> 2
```

Recall that `input()` always gives text. To do maths with a typed answer, convert it:

```
age_text = input("How old are you? ")
age = int(age_text)      # convert string to integer
print(age + 1)
```

You can also convert the other way, e.g. `str(12)` turns the number `12` into the text `"12"`, useful when combining numbers into a sentence using `+` instead of a comma.

Comparison operators ask a yes/no question about two values, and the answer is always a boolean:

```
score = 75
print(score > 50)    # True
print(score == 100)  # False
print(score != 100)  # True
```

Note: `==` (double equals) checks if two values are equal. A single `=` **stores** a value, they are not the same thing, and mixing them up is one of the most common beginner mistakes.

EX 5. First Example

```
score = 68
print(score > 50)
print(score >= 68)
print(score == 70)
```

Line-by-line:

- Line 1 stores 68 in `score`.
- Line 2 asks "is 68 greater than 50?" → `True`.
- Line 3 asks "is 68 greater than or equal to 68?" → `True`.
- Line 4 asks "does 68 equal 70?" → `False`.

TRY 6. Try It Yourself

Ask the user for two numbers using `input()`, convert both to integers, then print whether the first is greater than the second.

HINT 7. Hint

You'll need `int(input(...))` twice, then one comparison inside a `print()`.

CHECK 8. Check Yourself

1. What is `17 % 5`? Work it out before running it.
2. What's the difference between `=` and `==`?
3. Why does `input()` need `int()` before we can do maths with it?

DEBUG 9. Debugging Challenge

```
age = input("Enter your age: ")
next_year = age + 1
print(next_year)
```

What is wrong, and why does Python complain about combining a string and an integer?

CHALLENGE 10. Mini Challenge

Problem: Build a small “Number Checker.”

Requirements: Ask the user for a number, convert it to an integer, then print three separate results: whether it is greater than 10, whether it is exactly 0, and its remainder when divided by 2.

HINT Hint

You need one `input()`, one `int()` conversion, and three `print()` lines using comparison/arithmetic operators.

Optional harder challenge: Also print the result of dividing the number by 2 using `//` and using `/`, and describe in a comment why the two answers differ.

AI 11. AI/ML Connection

Machine learning models constantly ask comparison-style questions about data, “Is this measurement bigger than that threshold?”, to decide how to sort or classify something. The comparisons you just wrote by hand are, in a simplified way, exactly the kind of question a trained model asks automatically, thousands of times, once it has learned from data.

PRACTICE 12. Practice Tasks

- **Easy:** Print the result of `9 * 9`.
- **Easy:** Ask for a number and print whether it’s even (hint: use `%`).
- **Medium:** Ask for two numbers and print whether they are equal.
- **Medium:** Ask for someone’s height in centimeters and print whether it’s above 150.
- **Challenge:** Ask for a temperature in Celsius, and print whether it’s above 0, exactly 0, or below 0, as three separate comparisons (we’ll combine these into one decision next session).

EXIT 13. Exit Ticket

1. What symbol checks if two values are equal?
2. What does `//` do differently from `/`?
3. Why did the debugging challenge program crash?
4. What data type does a comparison always produce?

14. What You Should Be Able to Do Now

- ☐ I can use `+`, `-`, `*`, `/`, `//`, `%` to calculate values.
- ☐ I can convert between string, int, and float using `int()`, `float()`, `str()`.
- ☐ I can compare two values using `==`, `!=`, `>`, `<`, `>=`, `<=`.
- ☐ I understand that comparisons produce `True` or `False`.

SESSION 2, if, elif, else

GOAL 1. What You Will Learn

- use `if` to run code only when a condition is true
- use `else` to handle the opposite case
- use `elif` to check multiple conditions in order
- combine conditions using `and`, `or`, `not`

2. Why Does This Matter?

This is the moment your programs start **branching**, doing different things in different situations, instead of running the same steps every time. This is the single biggest building block between “a program that just displays things” and “a program that behaves intelligently.” Every classification decision

an ML model makes (“pass/fail,” “cat/dog,” “spam/not spam”) is, underneath, built from this same idea repeated automatically.

WORDS 3. Key Words

- **Condition:** A statement that is either `True` or `False`.
- **if statement:** Runs a block of code only if a condition is `True`.
- **else:** Runs a block of code when the `if` condition is `False`.
- **elif:** Short for “else if”, checks another condition if the first was `False`.
- **Logical operator:** `and`, `or`, `not`, combine or reverse conditions.

LEARN 4. Learn the Concept

```
score = 45
if score >= 50:
    print("You passed!")
else:
    print("You need to try again.")
```

Notice the **indentation** (the spaces before `print`), Python uses this to know which lines belong inside the `if`. This is not optional; it’s part of the language’s grammar.

To check several conditions in order, use `elif`:

```
score = 78
if score >= 80:
    print("Grade: A")
elif score >= 60:
    print("Grade: B")
elif score >= 40:
    print("Grade: C")
else:
    print("Grade: F")
```

Python checks each condition from top to bottom and stops at the first `True` one.

You can combine conditions:

```
age = 13
has_permission = True
if age >= 12 and has_permission:
    print("You may join the trip.")
```

`and` requires both sides to be `True`. `or` requires at least one side to be `True`. `not` flips `True` to `False` and vice versa.

EX 5. First Example

```
temperature = 15
if temperature > 30:
    print("It's hot outside.")
elif temperature > 15:
    print("It's warm outside.")
else:
    print("It's cool outside.")
```

Line-by-line: Python checks `temperature > 30`, false (15 is not greater than 30). It checks `temperature > 15`, false (15 is not greater than 15, it’s equal). None matched, so it runs the `else` block: “It’s cool outside.”

TRY 6. Try It Yourself

Write a program that asks the user for their exam score (0–100) and prints their grade using at least three `elif` branches.

HINT 7. Hint

Start from the highest grade boundary and work downward, like the temperature example.

CHECK 8. Check Yourself

1. What happens if none of the `if` / `elif` conditions are true and there's no `else` ?
2. In the grade example, why must we check `score >= 80` before `score >= 60` ?
3. What does `not True` evaluate to?

DEBUG 9. Debugging Challenge

```
score = 55
if score >= 50
    print("Pass")
else:
    print("Fail")
```

What is wrong with this program? *Hint: look carefully at the end of the `if` line.*

CHALLENGE 10. Mini Challenge

Problem: Build a “Pass or Fail” program.

Requirements: Ask for a student’s name and three scores (using `input()` + `int()`), calculate the average, then print whether the student passed (average ≥ 50), include the student’s name in the message.

HINT Hint

You’ll need to add the three scores and divide by 3 to get the average before your `if` statement.

Optional harder challenge: Add extra `elif` branches for “Distinction” (average ≥ 75) and “Needs Improvement” (average < 40).

AI 11. AI/ML Connection

A trained ML classifier is, in effect, an enormous, automatically-learned version of an `if` / `elif` / `else` chain, except instead of a human writing the thresholds (like “score ≥ 50 ”), the model *learns* the best thresholds itself from examples. Understanding how you’d write these rules by hand is exactly what will make Week 8’s explanation of “what a model learns” make sense.

PRACTICE 12. Practice Tasks

- **Easy:** Ask for a number and print “positive,” “negative,” or “zero.”
- **Easy:** Ask for someone’s age and print whether they can vote (18+).
- **Medium:** Build a “Weather Advisor”: ask for temperature and rain (yes/no as text), and give simple clothing advice using `and` / `or` .
- **Medium:** Build a simple password checker: ask for a password and print whether it’s at least 8 characters long (hint: `len(password)`).
- **Challenge:** Build a “Ticket Price” calculator: ask for age, and print the correct price using at least 3 age brackets (e.g. under 5 free, under 12 half-price, otherwise full price).

EXIT 13. Exit Ticket

1. What is the purpose of `elif` ?
2. Why does indentation matter in Python?
3. What’s the difference between `and` and `or` ?
4. Fix this in your head: what’s wrong with `if score = 50:` ?

14. What You Should Be Able to Do Now

- ☐ I can write an `if` / `else` statement.
- ☐ I can chain multiple conditions with `elif` .
- ☐ I can combine conditions with `and` , `or` , `not` .
- ☐ I understand why indentation matters in Python.

WEEKS 3–11, Continuing the Course

Weeks 3 to 11 follow the same lesson shape as Weeks 1 and 2, just written a little more compactly: objectives, a clear explanation with runnable code, a guided example, a try-it-yourself task with a hint, a debugging challenge, a mini-project, an AI/ML connection, and practice tasks, everything you need to work through the week on your own.

3 Repeating & Organizing Data (Loops and Lists)

GOAL Weekly Goal

Store many values at once in a list, and use loops to process every item without repeating code by hand.

CAREER Career Corner: On the Job as an AI/ML Engineer

Real datasets used by AI and Machine Learning engineers can contain millions of rows, far too many to look at one by one with your eyes. Engineers rarely process data by hand; instead, they write a loop once and let the computer repeat it perfectly, without getting tired or making a careless mistake. The pattern you are learning this week, store many values, then process every single one automatically, is the exact same pattern used to scan a million photographs while training an image-recognition model.

WORDS Key Words

List, index, **for** loop, **while** loop, iteration.

LEARN Concept

```
scores = [70, 85, 92, 61, 78]
print(scores[0]) # first item -> 70
print(scores[-1]) # last item -> 78
scores.append(100) # add a new item
print(len(scores)) # how many items -> 6
```

Lists are numbered starting at **0**, not 1, this trips up every beginner at least once, so expect it and don't worry when it happens.

LEARN Concept

```
scores = [70, 85, 92, 61, 78]
for score in scores:
    print(score)
```

This runs the indented block once for every item in the list, automatically. Combine with **if** :

```
for score in scores:
    if score >= 70:
        print(score, "- Pass")
    else:
        print(score, "- Fail")
```

LEARN Concept

```
count = 0
while count < 5:
    print("Count is", count)
    count = count + 1
```

while repeats *as long as* a condition stays **True**. Be careful: forgetting to change the condition variable inside the loop causes an **infinite loop**, a very common and very fixable beginner mistake (Ctrl+C or the Stop button in Thonny stops it).

EX Guided Example

```
scores = [70, 85, 92, 61, 78]
total = 0
for score in scores:
    total = total + score
average = total / len(scores)
print("Average score:", average)
```

TRY Try It Yourself

Given a list of 5 exam scores, print only the scores that are above the average.

HINT Hint

Calculate the average first (like the example above), then loop through the list again with an **if**.

DEBUG Debugging Challenge

```
scores = [70, 85, 92]
for score in scores
    print(score)
```

What's missing? (Same family of error as Week 2's **if** mistake, look for it.)

PROJECT Mini-Project

Ask the user to enter 5 scores one at a time using a **while** loop, store each in a list, then print: the total, the average, the highest score (hint: **max(scores)**), and the lowest (**min(scores)**).

AI AI/ML Connection

A dataset is essentially a very long list, often thousands or millions of rows. A **for** loop is how a computer looks at every single row, one at a time, which is exactly what happens (automatically, behind the scenes) when a model is trained or when we explore a dataset in Week 6–7.

PRACTICE Practice Tasks

- Easy: Print each item in a list of your 5 favorite foods.
- Easy: Count how many numbers in a list are even.
- Medium: Find the largest number in a list *without* using `max()`.
- Medium: Reverse a list by printing it from the last item to the first, using a loop and indexing.
- Challenge: Given a list of student names and a separate list of their scores (same order), print each name next to their score using a loop with an index.

WEEK 4, Reusable Code & Dictionaries (Functions)**GOAL** Weekly Goal

Write your own reusable functions, and store labeled data using dictionaries.

CAREER Career Corner: On the Job as an AI/ML Engineer

Open the code of any working AI or Machine Learning engineer, and you will see the same shape you are learning this week appearing again and again: a function that takes some input and returns a decision or a number. That is, in miniature, exactly what a trained Machine Learning model does. And dictionaries matter too, a great deal of real-world data arrives labeled and organized exactly like a dictionary, long before it is ever tidied into a clean spreadsheet.

WORDS Key Words

Function, parameter, argument, return value, dictionary, key, value.

LEARN Concept

```
def greet(name):  
    print("Hello,", name)  
  
greet("Amaka")  
greet("Tunde")
```

`def` defines a new function. `name` is a **parameter**, a placeholder for whatever value is passed in when the function is called. `"Amaka"` is the **argument**, the actual value used this time.

Functions can also **return** a value to be used later, instead of just printing:

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)
```

LEARN Concept

A dictionary stores data as labeled **key: value** pairs, much closer to how a real record (like one row of a dataset) looks:

```
student = {"name": "Zainab", "age": 13, "score": 88}
print(student["name"])
print(student["score"])
student["score"] = 91 # update a value
```

EX Guided Example

```
def describe_student(student):
    print(student["name"], "is", student["age"], "years old and scored", student["score"])

student1 = {"name": "Chinedu", "age": 12, "score": 76}
describe_student(student1)
```

TRY Try It Yourself

Write a function `is_passing(score)` that returns `True` if the score is 50 or above, and `False` otherwise. Test it with three different scores.

HINT Hint

Use `return`, not `print`, inside the function, then use `print()` when you call it.

DEBUG Debugging Challenge

```
def add(a, b)
    return a + b

print(add(2, 3))
```

Find the missing symbol (same family of mistake as loops and `if`, you're building the habit of spotting it).

PROJECT Mini-Project

Create a dictionary for each of 3 students (name, age, three scores as a list inside the dictionary). Write a function

`average_score(student)` that calculates and returns the average of a student's scores. Loop through all three students and print each one's name and average using your function.

AI AI/ML Connection

A dictionary is very close in shape to one row of a real dataset, a set of labeled features about one thing (one student, one house, one flower). A function that takes data in and gives a decision or number back out is, conceptually, exactly what a trained ML model *is*: something that takes features in and returns a prediction out.

PRACTICE Practice Tasks

- Easy: Write a function `square(n)` that returns a number multiplied by itself.
- Easy: Create a dictionary describing your favorite book (title, author, year) and print each value.
- Medium: Write a function `grade(score)` that returns "A", "B", "C" or "F" based on the score.
- Medium: Create a list of 3 dictionaries (3 students) and loop through printing each name and whether they passed, using your `is_passing()` function.
- Challenge: Write a function `class_average(students)` that takes a list of student dictionaries and returns the average of all their scores combined.

5 Files & Errors**GOAL** Weekly Goal

Save data to a file and load it back; understand and handle simple errors without panicking.

CAREER Career Corner: On the Job as an AI/ML Engineer

AI and Machine Learning engineers work with real files nearly every single day: downloaded datasets, saved trained models, logs recording what an AI system decided and why. Learning to open a file safely, read what is inside it, and calmly handle a file that might be missing or damaged is not a small side skill you can skip. It is a genuine, everyday part of the profession, and engineers who are careless about it lose data and waste hours.

WORDS Key Words

File, read, write, exception, `try / except` .

LEARN Concept

```
scores = [70, 85, 92]
file = open("scores.txt", "w")
for score in scores:
    file.write(str(score) + "\n")
file.close()
```

"w" means "write mode", this creates the file (or replaces it if it exists). `str(score)` converts the number to text, since files store text. `"\n"` moves to a new line.

LEARN Concept

```
file = open("scores.txt", "r")
for line in file:
    print(line.strip()) # .strip() removes the extra newline
file.close()
```

LEARN Concept

```
try:
    number = int(input("Enter a number: "))
    print(100 / number)
except ValueError:
    print("That wasn't a valid number.")
except ZeroDivisionError:
    print("You can't divide by zero.")
```

This lets your program respond calmly to a mistake instead of crashing. This is exactly what real data-handling code must do, since real-world data is often messy or incomplete.

EX Guided Example

Combine writing, reading, and error handling, save 3 scores typed by the user to a file, using `try / except` in case they type something that isn't a number.

TRY Try It Yourself

Write a program that reads `scores.txt` (created above) and prints the total of all the scores in it.

HINT Hint

Remember every line read from a file is text, you'll need `int()` on each line after `.strip()`.

DEBUG Debugging Challenge

```
file = open("scores.txt", "r")
data = file.read()
print(data)
```

What happens if `scores.txt` doesn't exist yet? Run it and read the error message carefully, what type of error is it? (This previews `FileNotFoundError`.)

PROJECT Mini-Project

Build a small program with two parts: (1) ask the user to type 5 scores and save them to `class_scores.txt`; (2) separately, read the file back, calculate and print the average, using `try / except` to handle the case where the file doesn't exist yet (print a friendly message instead of crashing).

AI AI/ML Connection

Every dataset used in Machine Learning lives in a file (very often a `.csv` file) before it's loaded into a program. What you just did, writing data out and reading it back in, is the exact same skill, at a small scale, that Week 6 uses to load a real dataset with Pandas.

PRACTICE Practice Tasks

- Easy: Write your name and school to a file called `me.txt`, then read it back and print it.
- Easy: Wrap a Week 2 age-input program in `try / except` so it doesn't crash on non-number input.
- Medium: Save a list of 5 favorite foods to a file, one per line.
- Medium: Write a program that reads a file of numbers and prints only the ones above 50.
- Challenge: Build a simple "logbook", each time the program runs, it *appends* (mode `"a"`) a new line with the date and a note, without erasing previous entries.

PHASE 3, PYTHON FOR DATA**6 Meet Your Data****GOAL** Weekly Goal

Understand what a dataset is, and load and look at a real one using Pandas.

CAREER Career Corner: On the Job as an AI/ML Engineer

This is one of the most important weeks in your entire journey toward becoming an AI or Machine Learning engineer. Ask any working engineer what they actually spend most of their time doing, and most will honestly say working with data, not writing clever algorithms. Understanding rows, columns, features, and targets is not a small preliminary step before the "real" work; for most engineers, it genuinely is most of the real work.

WORDS Key Words

Dataset, row, column, feature, label/target, structured data, CSV, DataFrame.

LEARN Concept

A dataset is simply a table: rows and columns, like a spreadsheet. Each **row** usually represents one example (one student, one house, one flower). Each **column** represents one piece of information about that example, we call each column a **feature**, except for one special column we're trying to predict, which we call the **label** or **target**.

name	study_hours	attendance	passed
Amaka	5	90	yes
Tunde	1	60	no
Zainab	4	95	yes

Here, `study_hours` and `attendance` are **features**. `passed` is the **target**, the thing we'd want a model to predict for a *new* student we don't already know the answer for.

FACT Did You Know?

Some datasets used by professional AI/ML engineers contain billions of rows and are far too large to fit into one computer's memory at once. Yet the underlying idea never changes, it is still just rows and columns, features and a target, exactly like the tiny table above. The tools change with scale; the core idea you are learning right now does not.

LEARN Concept

```
import pandas as pd
```

(Install once via Thonny's Manage Packages, as in the Setup Guide.) `pd` is just a short nickname we give the library so we don't have to type `pandas` every time.

LEARN Concept

```
import pandas as pd
data = pd.read_csv("students.csv")
print(data)
print(data.head())      # first 5 rows only
print(data.columns)     # column names
print(data.shape)       # (rows, columns)
```

EX Guided Example

Provide students a small ready-made `students.csv` (5–10 rows: name, study_hours, attendance, passed) to save into their week6 folder. Load it, print `.head()`, `.shape`, and `.columns`, and discuss what each row and column represents out loud.

TRY Try It Yourself

Using the same file, print just one column using `data["study_hours"]`, and print the number of rows using `len(data)`.

HINT Hint

Selecting a column uses square brackets with the column name in quotes, just like a dictionary key.

DEBUG Debugging Challenge

```
data = pd.read_csv("student.csv")
```

This crashes with `FileNotFoundError` even though `students.csv` exists. What's wrong? (Hint: read the filename very carefully, letter by letter.)

PROJECT Mini-Project

Load `students.csv`, print how many rows and columns it has, print the column names, print the first 3 rows, and write (in a comment) one sentence describing, in your own words, what each column means.

AI AI/ML Connection

This is the literal first step of nearly every real machine-learning project, get the data into your program and understand its shape before doing anything else with it. Skipping this step is one of the most common mistakes even professional data scientists make.

PRACTICE Practice Tasks

- Easy: Load the CSV and print only its column names.
- Easy: Print the last 3 rows using `data.tail(3)`.
- Medium: Print just the `attendance` column.
- Medium: Use `data["passed"].value_counts()` and explain in a comment what it shows.
- Challenge: Create your own tiny CSV by hand (open a text editor, type comma-separated rows, save as `.csv`) describing 5 of your favorite foods with columns like `name`, `price`, `is_healthy`, then load it with Pandas.

7 Exploring Data**GOAL** Weekly Goal

Calculate simple statistics about a dataset and draw a basic chart from it.

CAREER Career Corner: On the Job as an AI/ML Engineer

Before training a single model, professional AI and Machine Learning engineers explore their data first: checking averages, hunting for mistakes, and drawing simple charts to see what the numbers are actually saying. Skipping this step is one of the most common mistakes even experienced adults make when they are in a hurry, and it usually costs them far more time later. By practicing this habit now, in Week 7, you are already ahead of many beginners who jump straight to building a model.

WORDS Key Words

NumPy array, mean, Matplotlib, bar chart, line chart.

LEARN Concept

```
import numpy as np
scores = np.array([70, 85, 92, 61, 78])
print(scores.mean()) # average
print(scores.max())
print(scores.min())
```

NumPy is a library built for fast maths on lists of numbers, Pandas actually uses NumPy behind the scenes.

LEARN Concept

```
import pandas as pd
data = pd.read_csv("students.csv")
print(data["study_hours"].mean())
print(data["attendance"].max())
print(data.describe()) # summary of every numeric column at once
```

LEARN Concept

```
import matplotlib.pyplot as plt
import pandas as pd

data = pd.read_csv("students.csv")
plt.bar(data["name"], data["study_hours"])
plt.xlabel("Student")
plt.ylabel("Study Hours")
plt.title("Study Hours per Student")
plt.show()
```

EX Guided Example

Using `students.csv`, calculate the average `attendance`, then plot a bar chart of `attendance` per student, discussing what patterns are visible.

TRY Try It Yourself

Plot a bar chart of `study_hours` per student, and add a comment underneath stating which student studied the most and least, based only on looking at the chart.

HINT Hint

You need `plt.bar(x_values, y_values)`, then `plt.show()` at the end, don't forget `plt.show()` or nothing will appear.

DEBUG Debugging Challenge

```
plt.bar(data["name"], data["study_hours"])
```

Nothing appears when this is run. What's missing? (Hint: it's the last line of the guided example.)

PROJECT Mini-Project

Load `students.csv`, print `data.describe()`, then produce **two** charts: study hours per student, and attendance per student. Write two sentences (as comments) describing what each chart shows.

AI AI/ML Connection

Exploring data before building a model isn't optional, it's how real practitioners catch mistakes, spot patterns, and decide which features are worth using. The charts and averages you just produced are exactly what a data scientist does in the very first hour of a real project, before touching any ML code.

PRACTICE Practice Tasks

- Easy: Print the mean and max of the `attendance` column.
- Easy: Print `data.describe()` and identify (in a comment) the column with the highest average.
- Medium: Plot a bar chart comparing `passed` counts using `data["passed"].value_counts().plot(kind="bar")`.
- Medium: Create a NumPy array of 6 numbers of your choice and print its mean, max, and min.
- Challenge: Plot `study_hours` (x-axis) vs `attendance` (y-axis) as a scatter plot using `plt.scatter(data["study_hours"], data["attendance"])`, and describe in a comment whether there seems to be a relationship.

PHASE 4, INTRODUCTION TO AI AND MACHINE LEARNING

WEEK 8, What Is AI and Machine Learning, Really?

GOAL Weekly Goal

Build an accurate, non-magical mental model of what AI and ML actually are, using no new syntax, this is a concept-first week.

CAREER Career Corner: On the Job as an AI/ML Engineer

The vocabulary you are learning this week, model, training, features, labels, accuracy, overfitting, is not classroom-only language. It is the actual, working vocabulary used in real AI and Machine Learning engineering job interviews, research papers, team meetings, and everyday conversation between engineers. Learn these words well here, and you will genuinely understand what a working AI or Machine Learning engineer means when they talk about their work.

WORDS Key Words

Artificial Intelligence (AI), Machine Learning (ML), model, training, prediction, feature, target/label, training data, testing data, classification, regression, accuracy, overfitting.

LEARN Concept

Traditional programming (everything you've done Weeks 1–7):

Rules + Data → Answer

You wrote the rules (`if score >= 50: print("Pass")`). The computer just applies them.

Machine Learning flips this around:

Examples + Answers → Model

Instead of the human writing the rules, we give the computer many examples *with* their known answers, and the computer works out the rules (the **model**) itself.

Then, once trained:

New Data + Model → Prediction

We give the trained model a *new* example it has never seen, and it uses what it learned to predict an answer.

FACT Did You Know?

The term “machine learning” was first used all the way back in 1959, by a computer scientist named Arthur Samuel, who taught a computer to get better at playing checkers simply by having it play many games and learn from the results. The core idea you are learning this week, examples plus answers produce a model, is more than sixty years old. What has changed since then is mostly the amount of data and computing power available, not the basic idea itself.

LEARN Concept

- **Model:** A program the computer builds itself, by finding patterns in examples, rather than a human writing every rule directly.
- **Training:** The process of showing the model many examples so it can find patterns.
- **Feature:** An input the model uses to make its decision (e.g. `study_hours` , `attendance`).
- **Target/Label:** The answer we want the model to predict (e.g. `passed`).
- **Training data:** The examples used to teach the model.
- **Testing data:** A separate set of examples, *not* used in training, used afterward to check how well the model actually learned, like a test the model didn't see the answers for beforehand.
- **Classification:** Predicting a category (e.g. "pass" or "fail", "cat" or "dog").
- **Regression:** Predicting a number (e.g. a price, a score).
- **Accuracy:** Out of every prediction the model made on the testing data, what fraction was correct. If a model makes 8 correct predictions out of 10, its accuracy is 8/10, or 80%.
- **Overfitting:** When a model "memorizes" its training examples instead of learning a real pattern, it does very well on data it has seen before, but poorly on new data, the same way memorizing yesterday's test answers word-for-word won't help you on a different test covering the same topic.

Unplugged Activity, "Human as a Model": On paper (no computer), give students 8 example rows of `students.csv` -style data (study_hours, attendance, passed) as "training data," but hide the `passed` column on 2 new rows ("testing data"). Ask students to guess the missing answers using only the patterns they notice in the training rows. Discuss: what pattern did you use? Did everyone use the same pattern? How confident are you? This *is*, in miniature, exactly what a model does.

Guided Demo: Show `students.csv` again, and verbally walk through: "If I asked a computer to look for a pattern connecting study_hours and attendance to passed, without me writing an `if` statement, what do you think it would notice?" Connect this directly back to Week 6–7's exploration.

CHECK Check Yourself

1. In your own words, what is the difference between traditional programming and machine learning?
2. What is the difference between a feature and a target?
3. Why do we test a model on data it hasn't seen before, instead of the training data?
4. What might it mean if a model gets 100% accuracy on training data but very low accuracy on testing data?

AI AI/ML Connection

This entire lesson is the AI/ML connection, from here on, every remaining lesson will build directly on this vocabulary.

PRACTICE Practice Tasks

- Easy: In your own words, explain what "training" means.
- Easy: Give one example of a classification problem and one example of a regression problem, not used in this lesson.
- Medium: Explain why accuracy alone might be misleading if a dataset has 95 "no" answers and only 5 "yes" answers.
- Medium: Explain overfitting using an example from school life (not exams).
- Challenge: Sketch (on paper or in a text file) what features you think would help predict whether a football team wins a match.

PHASE 5, BUILDING AI/ML PROJECTS

9 Your First Model

GOAL Weekly Goal

Train, test, and use a real machine-learning model using scikit-learn, with heavy guidance.

CAREER Career Corner: On the Job as an AI/ML Engineer

What you are about to do this week, load data, split it, train a model, test it, and use it to make a prediction, is the exact daily workflow followed by professional AI and Machine Learning engineers, just at a much smaller and gentler scale. Professionals use far larger datasets and often more complicated models, but the six steps you are about to practice almost never change, no matter how advanced the project becomes.

WORDS Key Words

scikit-learn, train/test split, `.fit()`, `.predict()`, `.score()`.

FACT Did You Know?

The exact scikit-learn library you are about to use this week is not a simplified “student version.” It is the same real, professional library used inside many real companies for genuine classification and regression tasks, spam filters, recommendation systems, fraud detection, and much more. You are not practicing on a toy version of the tool, you are learning the real one, on a smaller, friendlier dataset.

LEARN Concept

```
pip install scikit-learn
```

(Or via Thonny’s Manage Packages, searching `scikit-learn`.)

LEARN Concept

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# 1. Load the data
data = pd.read_csv("students.csv")

# 2. Choose features (X) and target (y)
X = data[["study_hours", "attendance"]]
y = data["passed"]

# 3. Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Choose and train a model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# 5. Test the model
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)

# 6. Make a new prediction
new_student = [[3, 80]] # study_hours=3, attendance=80
prediction = model.predict(new_student)
print("Prediction:", prediction)
```

Explain each line as a direct continuation of Week 8’s vocabulary: `X` is the features, `y` is the target, `train_test_split` creates separate training and testing data, `.fit()` is training, `.score()` is testing/accuracy, `.predict()` is making a prediction on new, unseen data.

EX Guided Example

Walk through the full block above together, live, using the class's `students.csv` (expanded to at least 15–20 rows for this week so the split makes sense, provide this expanded file).

TRY Try It Yourself

Change `test_size` to `0.3` and re-run. Does the accuracy change? Try changing the `new_student` values and predict again with different study_hours/attendance combinations.

HINT Hint

You only need to change the numbers, not the structure of the code.

DEBUG Debugging Challenge

```
model = DecisionTreeClassifier()
model.predict(X_test)
```

This crashes. What critical step was skipped? (Hint: compare against the 6-step workflow above, one numbered step is entirely missing.)

PROJECT Mini-Project

Using the expanded `students.csv`, build the complete 6-step pipeline above from scratch (not copy-pasted, typed by hand, referring back to the example only as needed), then predict the outcome for 3 new made-up students, and write one sentence per prediction explaining, in plain language, what the model predicted and why that seems reasonable given the features.

AI AI/ML Connection

This is the AI/ML connection, you just trained a real machine-learning model using the same library and the same core method (though on a much smaller scale) that professional data scientists use.

PRACTICE Practice Tasks

- Easy: Re-run the pipeline changing only `random_state` to a different number, does the accuracy change?
- Easy: Print `X_train.shape` and `X_test.shape` and explain what the numbers mean.
- Medium: Add a third feature to `X` (if your CSV has one, e.g. `homework_done`) and see if accuracy changes.
- Medium: Try `RandomForestClassifier` instead of `DecisionTreeClassifier` (same `.fit` / `.predict` / `.score` pattern) and compare accuracy.
- Challenge: Predict outcomes for 5 new students with deliberately varied feature values, and write a short paragraph interpreting the pattern you notice in the model's predictions.

10 Build With Less Help**GOAL** Weekly Goal

Apply the same 6-step ML workflow to a new problem, with reduced step-by-step scaffolding, the bridge to full independence.

CAREER Career Corner: On the Job as an AI/ML Engineer

This week, you practice something real AI and Machine Learning engineers do constantly in their working lives: taking a workflow that worked well for one problem and thoughtfully adapting it to a brand-new one. This kind of flexible, transferable thinking, rather than memorizing a single fixed project, is genuinely what separates someone who can only follow a tutorial from someone who can actually build.

WORDS Key Words

Regression, `LinearRegression`, `mean_absolute_error`, feature/target selection (recap).

LEARN Concept

When the target is a *number* instead of a category, we use regression:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

data = pd.read_csv("students.csv")
X = data[["study_hours", "attendance"]]
y = data["exam_score"] # a number this time, not yes/no

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
error = mean_absolute_error(y_test, predictions)
print("Average prediction error:", error)

new_prediction = model.predict([[4, 85]])
print("Predicted score:", new_prediction)
```

Note the workflow is **the same 6 steps as Week 9**, only the type of model and the way we measure success change (accuracy doesn't make sense for numbers, so we use average error instead: "on average, our predictions were off by about this many points").

Guided-but-lighter walkthrough: Rather than typing every line together as a class, students are given the 6-step structure as headings only (Load → Choose features/target → Split → Train → Test → Predict) and asked to fill in each step themselves, checking against the reference code only when stuck.

TRY Try It Yourself

Using a provided dataset (choose one, offline, appropriate for a laptop: e.g. a small "fruit weight/size → fruit type" classification set, or the classic small Iris flower dataset which ships with scikit-learn via `from sklearn.datasets import load_iris`), decide for yourself which columns should be features and which should be the target, then build the full pipeline.

HINT Hint

Ask yourself: "What am I trying to predict?", that column is your target; everything else useful becomes a feature.

DEBUG Debugging Challenge

Give students a pipeline with the split and training steps swapped in order (`model.fit()` called before `train_test_split()`), and ask them to find and explain the problem.

PROJECT Mini-Project

Offer 2–3 small prepared datasets (see Part 5). Students choose one, and independently complete the full 6-step pipeline, writing a short explanation of their feature/target choice *before* coding it (this rehearses the decision-making the final project will require).

AI AI/ML Connection

This week proves the workflow from Week 9 wasn't a one-off, the same shape (load → choose features/target → split → train → test → predict) applies to a completely different problem and a completely different type of model. That reusable shape is exactly what the final project will ask students to apply on their own.

PRACTICE Practice Tasks

- Easy: Change the features used and see how the error changes.
- Easy: Predict the value for 3 new, self-invented inputs and sanity-check whether the numbers seem reasonable.
- Medium: Compare `LinearRegression` against `DecisionTreeRegressor` (same interface) on the same data.
- Medium: Write, in your own words, why we can't use "accuracy" for a regression problem.
- Challenge: Pick a completely different pair of features from the same dataset and see whether the prediction error gets better or worse, explain why that might be.

PHASE 6, INDEPENDENT FINAL PROJECT

11 Your Own Project

GOAL Weekly Goal

Independently design, build, debug, test, and present a small machine-learning project from start to finish.

CAREER Career Corner: On the Job as an AI/ML Engineer

By completing this final project, you join a small and genuinely capable group of people: those who have taken a real problem, gathered data for it, trained a model on it, tested whether it worked, and explained the results clearly to someone else. That exact loop, problem, data, model, test, explain, is the loop a professional AI or Machine Learning engineer repeats their entire career, simply with bigger problems, bigger teams, and bigger datasets. You have now done it for real, yourself, from the ground up.

This week has **no new syntax**, every skill needed already exists from Weeks 1–10. This week is about applying everything you already know, not learning something new.

Session 1, Design and Build: Students work through the **Final Project Framework** (Part 5) independently:

1. Choose a problem (from the provided project list, or their own idea, teacher-approved).
2. Identify what data is needed.
3. Choose or create a small dataset.
4. Load the data.
5. Explore the data (shape, columns, simple stats/chart).
6. Choose features and target, and write one sentence justifying the choice.
7. Split the data into training and testing sets.
8. Train a model.
9. Evaluate it (accuracy or error).
10. Make at least 2 new predictions and interpret them in plain language.

Session 2, Improve, Test, and Present:

11. Improve or experiment (try a different feature set, a different model, or more data) and compare results.
12. Present the project to another student or the class: what problem it solves, what data it uses, how the model performs, and what a new prediction means, without reading from a script.

Debugging & Support: Students keep a short “Bug Log”, every error encountered, what it meant, and how it was fixed, reinforcing Week 1’s error-first mindset one last time, now applied to their own original work.

AI AI/ML Connection

This *is* the connection, the entire course was designed so that, by this point, a student has already independently practiced every single piece this final project requires.

PART 4, EXERCISES AND SOLUTIONS

Exercises for each week are listed in Part 3 under “Try It Yourself,” “Mini Challenge,” and “Practice Tasks.” Solutions are provided here, separated from the exercises, so students can attempt each task before checking. Use the progression: **Task** → **Think** → **Hint (given in Part 3)** → **Try Again** → **Solution (here)**.

Week 1 Solutions

DEBUG Debugging Challenge

Missing closing parenthesis. Corrected:

```
print("Hello, World!")
```

CHALLENGE Mini Challenge

```
# My About Me program
print("My name is Musa")
print("I am 13 years old")
print("My favorite subject is Basic Science")
print("I am learning Python this year")
```

Week 2 Solutions

DEBUG Debugging Challenge

`age` is a string from `input()` ; adding `1` (an int) to it fails. Fix:

```
age = int(input("Enter your age: "))
next_year = age + 1
print(next_year)
```

DEBUG Debugging Challenge

Missing colon after `if score >= 50` . Fix:

```
score = 55
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

CHALLENGE Mini Challenge

```
name = input("Enter your name: ")
s1 = int(input("Enter score 1: "))
s2 = int(input("Enter score 2: "))
s3 = int(input("Enter score 3: "))
average = (s1 + s2 + s3) / 3
if average >= 50:
    print(name, "passed with an average of", average)
else:
    print(name, "did not pass. Average was", average)
```

Week 3 Solutions**DEBUG** Debugging Challenge

Missing colon after `for score in scores` . Try It Yourself (above-average scores):

```
scores = [70, 85, 92, 61, 78]
average = sum(scores) / len(scores)
for score in scores:
    if score > average:
        print(score)
```

Week 4 Solutions**DEBUG** Debugging Challenge

Missing colon after `def add(a, b)` . Try It Yourself (is_passing):

```
def is_passing(score):
    return score >= 50

print(is_passing(72))
print(is_passing(40))
print(is_passing(50))
```

Week 5 Solutions**TRY** Try It Yourself

```
file = open("scores.txt", "r")
total = 0
for line in file:
    total = total + int(line.strip())
file.close()
print("Total:", total)
```

Weeks 6–10 Solutions

Because these weeks work with datasets that vary by the specific file each class uses, model solutions are provided as fully worked code inside the “Guided Example” and “Mini-Project” sections of Part 3 for each week, the pattern shown there is the solution pattern; students substitute their own column names and file names.

Week 11

There is no fixed “solution” for the final project by design, evaluation uses the rubric in Part 6.

PART 5, PROJECTS

Weekly Mini-Projects

(Full details for each are in Part 3 under each week's "Mini-Project"/"Mini Challenge" section.)

1. Week 1, About Me program
2. Week 2, Pass or Fail grader
3. Week 3, Class Score Tracker
4. Week 4, Student Record System
5. Week 5, Save and Load Scores
6. Week 6, Dataset exploration report
7. Week 7, Two-chart data exploration
8. Week 8, "Human as a Model" reflection
9. Week 9, Guided Pass/Fail Predictor
10. Week 10, Choose Your Own Guided Project

Final Project Framework (for all Final Project options)

Step 1. Choose a problem. **Step 2.** Identify the data needed. **Step 3.** Choose or create a dataset. **Step 4.** Load the data. **Step 5.** Explore the data. **Step 6.** Choose features and target. **Step 7.** Split the data. **Step 8.** Train the model. **Step 9.** Evaluate it. **Step 10.** Make predictions. **Step 11.** Improve or experiment with it. **Step 12.** Present the project.

Final Project Options (choose one, or propose your own using the same framework)

Option A, Will I Pass? (Classification)

- **Problem statement:** Predict whether a student passes or fails based on study-related habits.
- **Required skills:** Weeks 1–9 (variables through classification).
- **Possible dataset:** A small class-collected or provided CSV (`study_hours` , `attendance` , `homework_done` , `passed`), as few as 20 rows is enough.
- **Input/features:** `study_hours` , `attendance` , `homework_done` .
- **Target:** `passed` (yes/no).
- **Expected output:** A predicted pass/fail for a new, made-up student.
- **Model type:** Classification (`DecisionTreeClassifier`).
- **Difficulty:** Beginner-friendly, recommended default option.
- **Extension ideas:** Add more features (e.g. sleep hours); compare two different classifier types.

Option B, What Type of Flower Is This? (Classification, classic small dataset)

- **Problem statement:** Predict which species a flower belongs to from its measurements.
- **Required skills:** Weeks 1–9.
- **Possible dataset:** The built-in Iris dataset (`from sklearn.datasets import load_iris`), no internet or file needed, runs fully offline, tiny and fast on any laptop.
- **Input/features:** Petal length, petal width, sepal length, sepal width.
- **Target:** Flower species.
- **Expected output:** Predicted species for new measurements.
- **Model type:** Classification.

- **Difficulty:** Beginner-friendly.
- **Extension ideas:** Visualize the data with a scatter plot before training; try predicting with only 2 of the 4 features and compare accuracy.

Option C, Predicting Exam Scores (Regression)

- **Problem statement:** Predict a numeric exam score from study habits.
- **Possible dataset:** Same class CSV as Option A, with an added `exam_score` numeric column.
- **Input/features:** `study_hours` , `attendance` .
- **Target:** `exam_score` (a number).
- **Expected output:** A predicted score for a new student.
- **Model type:** Regression (`LinearRegression`).
- **Difficulty:** Beginner–intermediate.
- **Extension ideas:** Compare Linear Regression against a Decision Tree Regressor.

Option D, Fruit or Vegetable Classifier (Classification, very small dataset)

- **Problem statement:** Classify an item as fruit or vegetable based on simple measurable properties.
- **Possible dataset:** A tiny, hand-typed CSV of ~15 rows (`weight_grams` , `is_sweet` (1/0), `has_seeds_outside` (1/0), `type`), ideal for students with limited computing resources, since it can be typed by hand in 10 minutes with no download needed.
- **Input/features:** `weight_grams` , `is_sweet` , `has_seeds_outside` .
- **Target:** `type` (fruit/vegetable).
- **Model type:** Classification.
- **Difficulty:** Beginner-friendly, recommended for very limited-resource situations.
- **Extension ideas:** Add more example rows collected as a class; add a third category.

Option E, Simple Movie/Book Recommendation (Beginner Recommendation Logic)

- **Problem statement:** Suggest whether someone would probably enjoy a book/movie based on genre-preference patterns.
 - **Possible dataset:** A small hand-built CSV of ~20 rows (`genre` , `length_minutes_or_pages` , `liked` (yes/no)) built from real class preferences.
 - **Input/features:** `genre` (converted to numbers, a light preview of encoding categories, explained simply as “giving each genre a number so the computer can compare them”), `length` .
 - **Target:** `liked` .
 - **Model type:** Classification.
 - **Difficulty:** Intermediate (introduces one small new idea, encoding text categories as numbers, explained step by step when needed).
 - **Extension ideas:** Build it from the class’s own real survey data collected as a group activity.
-

PART 6, ASSESSMENTS

Weekly Exercises & Quizzes

Each week's "Check Yourself" and "Exit Ticket" sections (Part 3) function as short formative quizzes (3–5 questions), used at the start of the next session as a review warm-up.

Sample Mini-Quiz Format (used at the start of Sessions, 5 minutes)

Example, Week 3 Session 2 warm-up quiz:

1. What does `scores[0]` return in a list `scores = [10, 20, 30]` ?
2. What is the difference between `for` and `while` ?
3. What would happen if a `while` loop's condition never becomes `False` ?

Coding Challenges

The "Challenge" item in each week's Practice Tasks (Part 3) serves as that week's coding challenge, gradually increasing in required independence.

Mid-Course Practical Assessment (End of Week 6)

A 45-minute, closed-notes-but-open-own-code practical:

1. Write a program using a list, a loop, and an `if` statement to process a small dataset of 5 scores (typed in the assessment) and print who passed/failed. (40%)
2. Load a provided small CSV with Pandas and print its shape, columns, and the average of one numeric column. (40%)
3. Explain, in 2–3 sentences, what a "feature" and a "target" mean, using an example from the CSV. (20%)

Final Project Assessment

Combines the completed project (code + short write-up) and a short live or recorded explanation (2–4 minutes) of the project.

Final-Project Rubric (100 points total)

Criterion	Weight	What "Strong" Looks Like
Problem understanding	15	Student can clearly state, in their own words, what problem the project solves and why it matters
Python code quality	15	Code runs, is reasonably organized, uses appropriate variables/functions/loops from the course
Data handling	15	Data is loaded correctly; student can describe what each feature and the target represent
ML implementation	20	Correct use of the 6-step workflow: split, train, test, predict
Testing/evaluation	10	Student reports and correctly interprets accuracy or error
Experimentation	10	Student made at least one deliberate change (feature, model, data) and compared results
Explanation	10	Student can explain the project to someone else,

		unscripted, in plain language
Independence	5	Project reflects the student's own decisions, not a copied tutorial

Presentation polish (slides, visuals) is **not** separately scored, it may support the “Explanation” criterion only if it aids clarity, never as a criterion on its own, per the course’s technical-understanding-first principle.

PART 7, FINAL PROJECT READINESS MATRIX

Final Project Skill	Week Introduced	Week(s) Practiced	Final Project Use
Variables	Week 1	Weeks 1–11 (continuously)	Storing inputs, results, and predictions
Conditions (<code>if</code> / <code>elif</code> / <code>else</code>)	Week 2	Weeks 2, 3, 9–11	Interpreting predictions, branching logic in exploration
Lists	Week 3	Weeks 3, 4, 6–11	Holding features, scores, results
Loops	Week 3	Weeks 3–11	Iterating through data/rows
Functions	Week 4	Weeks 4, 9–11	Reusable prediction/interpretation helpers
Dictionaries	Week 4	Weeks 4, 6	Understanding row/record structure before Pandas
Files (read/write)	Week 5	Weeks 5, 6	Loading/saving datasets
Error handling (try/except)	Week 5	Weeks 5, 9–11	Handling bad input/missing files gracefully
Datasets/rows/columns	Week 6	Weeks 6–11	Understanding the project's data
Pandas	Week 6	Weeks 6, 7, 9–11	Loading and inspecting the dataset
Data visualization	Week 7	Weeks 7, 11	Exploring data before modeling
Features/target	Week 6 (intro), Week 8 (formalized)	Weeks 8, 9, 10, 11	Selecting X and y
Train/test split	Week 9	Weeks 9, 10, 11	Splitting the project's dataset
ML model (fit)	Week 9	Weeks 9, 10, 11	Training the project's model
Evaluation (score/error)	Week 9	Weeks 9, 10, 11	Reporting how good the project's model is
Prediction	Week 9	Weeks 9, 10, 11	Producing and interpreting new predictions

Readiness check: every row above shows the skill introduced at least one full week before Week 9 (the first week it's required for real model-building), and practiced at least once more before Week 11. No skill required by the final project is introduced for the first time during Weeks 10–11. This satisfies the course's core design test: a student who has completed Weeks 1–10 in full has already personally practiced, at least once, every single skill the final project requires.

Closing Note From Mr. Muwatta

If you have followed this material week by week, from your very first `print("Hello, World!")` to your own finished Machine Learning project, you have done something that many people twice your age have never done. Be proud of that.

Keep exploring, keep experimenting, and keep asking “why” whenever your code does not do what you expected. That question is the whole of programming.

Mr. Muwatta

Founder, Algorise Tech Explorers

www.muwatta.com.ng